

UNIDAD DE TRABAJO 02 · PERSISTENCIA RELACIONAL

# Manejo de Conectores (JDBC y Oracle Database)

Arquitectura JDBC 4.3, Sentencias Parametrizadas, Procedimientos Almacenados, Transacciones ACID, Inspección de Metadatos, Pools de Conexiones de Alto Rendimiento (HikariCP) y Patrones DAO / DTO

MÓDULO CURRICULAR

**Acceso a Datos (0486)**

DOCENTE

**Sergio Capdevila Díez**

CICLO FORMATIVO

**2º DAM (Desarrollo Multiplataforma)**

DURACIÓN ESTIMADA

**35 Horas Lectivas (Trimestre 1)**

RESULTADOS DE APRENDIZAJE

**RA2 (CE2.a - CE2.h)**

STACK TECNOLÓGICO

**Java 21 · Oracle Database 23c · Maven**

# 1. ¿Qué es JDBC y cuál es su Arquitectura?

Capa de abstracción estándar entre el código Java y los motores RDBMS relacionales

## Arquitectura en Capas JDBC

ESTÁNDAR JAVA SE

JDBC (*Java Database Connectivity*) desacopla la aplicación de las particularidades del motor SQL:

- **Java Application:** Invoca interfaces estándar del paquete `java.sql`.
- **JDBC Driver Manager:** Gestiona y enruta llamadas al conector adecuado.
- **JDBC Driver (Vendor):** Implementa el protocolo binario del motor (Oracle, PostgreSQL, etc.).
- **Database Server:** Ejecuta consultas y gestiona el almacenamiento transaccional.

## 💡 ANALOGÍA COTIDIANA: EL ADAPTADOR DE CORRIENTE UNIVERSAL

Imagina viajar por todo el mundo con tus dispositivos electrónicos. Cada país tiene un enchufe de pared diferente (Oracle, PostgreSQL, MySQL), pero tu cargador tiene siempre la misma clavija (tu código Java).

El **Driver JDBC** es el adaptador físico específico para cada país: la clavija de tu ordenador nunca cambia, pero el conector traduce al voltaje y geometría del enchufe local.

## 🚀 EL CONECTOR ORACLE TYPE 4 (PURE JAVA THIN DRIVER)

En nuestro módulo utilizamos el driver oficial de Oracle **ojdbc11**. Es un conector de tipo 4: código 100% Java puro que se comunica directamente por TCP/IP con el puerto 1521 de Oracle sin requerir instalar clientes nativos C/C++ en el sistema operativo.

## 2. Estableciendo Conexión con Oracle Database

Dependencia Maven, cadena de conexión (JDBC URL) y DriverManager

### Configuración Maven (pom.xml)

COM.ORACLE.DATABASE.JDBC

```
<dependency>
  <groupId>com.oracle.database.jdbc</groupId>
  <artifactId>ojdbc11</artifactId>
  <version>23.3.0.23.09</version>
</dependency>
```

En JDBC 4.0+ ya **no es obligatorio** invocar  
`Class.forName("oracle.jdbc.OracleDriver")` gracias al  
mecanismo `ServiceLoader` de Java.

### Sintaxis de Conexión en Java

JAVA.SQL.DRIVERMANAGER

```
String url = "jdbc:oracle:thin:@localhost:1521:xe";
String user = "c##dam2_user";
String pass = "dam2_pass";

try (Connection conn = DriverManager.getConnection(url, user, pass)) {
  System.out.println("¡Conexión establecida: "
    + conn.getMetaData().getDatabaseProductVersion());
}
```

#### ⚠ PARTICULARIDAD CRÍTICA DE ORACLE DATABASE 12C, 19C Y 23C

Al crear usuarios comunes en la base de datos raíz (CDB), Oracle exige obligatoriamente que los nombres comiencen por el prefijo `c##` (ejemplo: `c##dam2_user`). Si olvidas este prefijo al crear el usuario en SQL\*Plus, Oracle lanzará un error `ORA-65096`.

### 3. Statement vs PreparedStatement

Por qué la concatenación de texto en sentencias SQL es una práctica prohibida

#### ❌ Statement (Peligroso)

VULNERABLE A INYECCIÓN  
SQL

```
// Peligro: inyección SQL inmediata
String sql = "SELECT * FROM usuarios WHERE email = '"
            + txtEmail + "' AND pass = '" + txtPass + "'";
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sql);
```

Si el usuario ingresa admin' OR '1'='1, la consulta altera su semántica y devuelve todos los registros saltándose la autenticación.

#### ✅ PreparedStatement (Seguro y Rápido)

COMPILACIÓN  
PREVIA

```
// Seguro: parámetros posicionales '?'
String sql = "SELECT * FROM usuarios WHERE email = ? AND pass = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, txtEmail);
    ps.setString(2, txtPass);
    try (ResultSet rs = ps.executeQuery()) { ... }
}
```

El motor compila y optimiza el árbol de ejecución una sola vez. Los valores entran como parámetros tipados sin alterar el código SQL.

#### 🚀 VENTAJA DOBLE: SEGURIDAD ABSOLUTA + CACHÉ DE EJECUCIÓN EN ORACLE

Oracle almacena los planes de ejecución en la *Shared Pool (SGA)*. Usar PreparedStatement permite a Oracle reutilizar el plan existente para miles de llamadas con parámetros distintos (*Soft Parse*), disparando el rendimiento hasta un 400%.

## 4. Operaciones CRUD con PreparedStatement

Ejecución de INSERT, UPDATE, DELETE y recuperación de Claves Generadas

### Inserción con Claves Autogeneradas

**RETURN\_GENERATED\_KEYS**

```
String sql = "INSERT INTO productos (nombre, precio, stock) "
            + "VALUES (?, ?, ?)";
try (PreparedStatement ps = conn.prepareStatement(sql,
        Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, "Portátil HP");
    ps.setBigDecimal(2, new BigDecimal("850.50"));
    ps.setInt(3, 15);

    int filas = ps.executeUpdate(); // Retorna filas afectadas

    try (ResultSet rs = ps.getGeneratedKeys()) {
        if (rs.next()) long id = rs.getLong(1);
    }
}
```

### Actualización y Borrado

**EXECUTEUPDATE()**

```
String sql = "UPDATE productos SET precio = ?, stock = stock + ? "
            + "WHERE id_producto = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setBigDecimal(1, nuevoPrecio);
    ps.setInt(2, unidades);
    ps.setLong(3, id);

    int actualizadas = ps.executeUpdate();
    if (actualizadas == 0) {
        throw new NoSuchElementException("Producto no hallado: " + id);
    }
}
```



#### REGLA MNEMOTÉCNICA DE MÉTODOS EN JDBC

**executeQuery():** Exclusivo para consultas SELECT (retorna un ResultSet).

**executeUpdate():** Para sentencias DML (INSERT, UPDATE, DELETE) y DDL (CREATE, ALTER). Retorna un entero int indicando el recuento de filas modificadas.

## 5. Navegación y Lectura con ResultSet

El cursor de base de datos, métodos getters tipados y el tratamiento de nulos

### Bucle Típico de Lectura

RS.NEXT()

```
String sql = "SELECT id, nombre, precio, fecha_alta FROM prod";
try (PreparedStatement ps = conn.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        long id = rs.getLong("id");
        String nom = rs.getString("nombre");
        BigDecimal precio = rs.getBigDecimal("precio");
        LocalDate fecha = rs.getObject("fecha_alta", LocalDate.class);

        System.out.println(id + " | " + nom + " | " + precio);
    }
}
```

### El Peligro de los NULLs y wasNull() VALORES PRIMITIVOS

```
int departamento = rs.getInt("id_departamento");
// Si el valor era NULL en la BD, Java retorna 0.
// Para verificar si era realmente null:
if (rs.wasNull()) {
    System.out.println("El empleado no tiene departamento asignado.");
}

// En Java Moderno (Java 8+):
Integer dep = rs.getObject("id_departamento", Integer.class);
// Retorna null de forma natural y limpia.
```

#### ● ERROR TÍPICO: INDEXOUTOFBOUNDSEXCEPTION EN JDBC

A diferencia de los arrays y listas en Java que empiezan en el índice 0, las columnas y parámetros en JDBC se indexan empezando en 1. Si ejecutas `rs.getString(0)` o `ps.setString(0, ...)`, la JVM lanzará un `SQLException`: Índice de columna no válido.

## 6. Invocación de Procedimientos PL/SQL

Uso de CallableStatement con parámetros de entrada (IN) y salida (OUT)

### Procedimiento Almacenado en Oracle PL/SQL DDL

```
CREATE OR REPLACE PROCEDURE sp_calcular_descuento (  
    p_id_cliente IN NUMBER,  
    p_total_compras OUT NUMBER,  
    p_porcentaje_desc OUT NUMBER  
) AS  
BEGIN  
    SELECT NVL(SUM(importe), 0) INTO p_total_compras  
    FROM pedidos WHERE id_cliente = p_id_cliente;  
  
    IF p_total_compras > 1000 THEN p_porcentaje_desc := 15;  
    ELSE p_porcentaje_desc := 5;  
    END IF;  
END;
```

### Invocación en Java

CALLABLESTATEMENT

```
String sql = "{ call sp_calcular_descuento(?, ?, ?) }";  
try (CallableStatement cs = conn.prepareCall(sql)) {  
    cs.setLong(1, 105L); // Parámetro IN  
  
    // Registrar parámetros OUT indicando el tipo SQL  
    cs.registerOutParameter(2, Types.NUMERIC);  
    cs.registerOutParameter(3, Types.NUMERIC);  
  
    cs.execute(); // Ejecutar  
  
    // Leer resultados  
    double total = cs.getDouble(2);  
    int descuento = cs.getInt(3);  
}
```

### ★ ¿CUÁNDO USAR CALLABLESTATEMENT EN EL MUNDO PROFESIONAL?

Cuando las operaciones de cálculo masivo o transferencias bancarias residen directamente en el servidor de base de datos por rendimiento, seguridad o cumplimiento normativo de la organización, evitando transferir millones de registros por la red hasta la aplicación Java.

## 7. Retorno de Conjuntos con SYS\_REFCURSOR

Cómo devolver colecciones de datos completas desde funciones o paquetes Oracle

### Función PL/SQL con SYS\_REFCURSOR

ORACLE  
23C

```
CREATE OR REPLACE FUNCTION fn_listar_empleados_dep(
  p_dep_id IN NUMBER
) RETURN SYS_REFCURSOR AS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR
    SELECT id, nombre, salario
    FROM empleados
    WHERE id_departamento = p_dep_id;
  RETURN v_cursor;
END;
```

### Consumo en JDBC con Types.REF\_CURSOR

JAVA  
21

```
String sql = "{ ? = call fn_listar_empleados_dep(?) }";
try (CallableStatement cs = conn.prepareCall(sql)) {
  // Parámetro 1: Retorno de la función (Cursor)
  cs.registerOutParameter(1, Types.REF_CURSOR);
  cs.setInt(2, 10); // Departamento 10

  cs.execute();

  // Convertir el cursor devuelto a un ResultSet
  try (ResultSet rs = (ResultSet) cs.getObject(1)) {
    while (rs.next()) {
      System.out.println(rs.getString("nombre"));
    }
  }
}
```

#### ⚡ ESTÁNDAR JDBC 4.2 VS CAST NATIVO

En Java 8+ / JDBC 4.2+, `Types.REF_CURSOR` está estandarizado en el paquete `java.sql.Types`. Antes requería `OracleTypes.CURSOR` de la librería privativa. Ahora el código es portable y limpio.



## 8. Gestión Transaccional ACID en JDBC

Control manual de transacciones con `setAutoCommit(false)`, `commit()` y `rollback()`

### Las 4 Propiedades ACID

BASES DE DATOS ROBUSTAS

- **Atomicidad (All-or-Nothing):** O se ejecutan todas las operaciones con éxito, o ninguna surte efecto.
- **Consistencia:** La base de datos pasa de un estado válido a otro cumpliendo restricciones.
- **Aislamiento:** Las operaciones simultáneas no interfieren entre sí.
- **Durabilidad:** Una vez confirmado el commit, los cambios sobreviven a caídas eléctricas.

### Patrón Transaccional Canónico

TRY-CATCH-FINALLY

```
try (Connection conn = dataSource.getConnection()) {
    conn.setAutoCommit(false); // 1. Desactivar auto-confirmación
    try {
        debitarCuenta(conn, origenId, importe);
        acreditarCuenta(conn, destinoId, importe);

        conn.commit(); // 2. Confirmación atómica
    } catch (Exception e) {
        conn.rollback(); // 3. Deshacer en caso de error
        throw e;
    } finally {
        conn.setAutoCommit(true); // 4. Restaurar
    }
}
```

### ⚠ EL PELIGRO DEL MODO AUTO-COMMIT POR DEFECTO

Por defecto, cualquier conexión JDBC tiene `autoCommit = true`. Esto significa que cada `executeUpdate()` confirma inmediatamente el cambio en disco. Si en una transferencia bancaria falla el ingreso tras haber retirado el saldo, ¡el dinero desaparecerá en el limbo si no gestionas la transacción manualmente!

## 9. Puntos de Retorno Parcial: Savepoints

Deshacer operaciones intermedias sin perder el trabajo previo de la transacción

### ¿Qué es un Savepoint?

JAVA.SQL.SAVEPOINT

Un Savepoint actúa como un punto de restauración temporal dentro de una transacción activa:

- `conn.setSavepoint("nombre")`: Marca el punto actual.
- `conn.rollback(sp)`: Deshace únicamente las operaciones posteriores al punto.
- `conn.releaseSavepoint(sp)`: Libera los recursos del punto intermedio.

### Ejemplo: Proceso por Lotes con Fallo Parcial

RESILIENCIA

```
conn.setAutoCommit(false);
Savepoint sp = conn.setSavepoint("PuntoClienteA");
try {
    procesarCobroCliente(conn, clienteA);
} catch (PaymentException pe) {
    // Solo deshacemos el cliente A, manteniendo el resto
    conn.rollback(sp);
    registrarFallo(clienteA);
}
// Continuamos con el resto de clientes del lote...
conn.commit();
```

#### 💡 ANALOGÍA COTIDIANA: PUNTOS DE GUARDADO EN VIDEOJUEGOS

Imagina jugar un nivel largo de un videojuego. Antes de enfrentarte a una trampa peligrosa, tocas un *CheckPoint* (Savepoint). Si mueres en la trampa, reapareces en ese punto sin tener que empezar el juego entero desde el principio (rollback total).

# 10. Inspección de Metadatos del Motor

DatabaseMetaData y ResultSetMetaData: conociendo la BD en tiempo de ejecución

## DatabaseMetaData (Catálogo)

CONN.GETMETADATA()

Permite conocer las capacidades del motor, versión, soporte transaccional y tablas del usuario:

```
DatabaseMetaData dmd = conn.getMetaData();
System.out.println("Motor: " + dmd.getDatabaseProductName());
System.out.println("Soporta Savepoints: " + dmd.supportsSavepoints());

// Listar tablas de la base de datos
try (ResultSet rs = dmd.getTables(null, "C##DAM2_USER", "%", new String[]{"TABLE"})) {
    while (rs.next()) System.out.println("Tabla: " + rs.getString("TABLE_NAME"));
}
```

## ResultSetMetaData (Columnas)

RS.GETMETADATA()

Describe la estructura dinámica del resultado de cualquier consulta SELECT:

```
ResultSetMetaData rsmd = rs.getMetaData();
int columnas = rsmd.getColumnCount();

for (int i = 1; i <= columnas; i++) {
    System.out.printf("Col %d: %-15s Tipo: %s (%d)\n",
        i,
        rsmd.getColumnLabel(i),
        rsmd.getColumnTypeName(i),
        rsmd.getColumnDisplaySize(i));
}
```

### 🔧 UTILIDAD EN HERRAMIENTAS PROFESIONALES

¿Cómo funcionan herramientas como DBeaver, DataGrip o SQL Developer para mostrar dinámicamente tablas en un árbol gráfico? ¿Utilizan exactamente las interfaces DatabaseMetaData y ResultSetMetaData!

# 11. El Problema del DriverManager a Demanda

Por qué abrir y cerrar conexiones en cada petición de usuario colapsa el servidor

## El Coste Oculto de una Conexión

~100-300 MS POR  
HANDSHAKE

Cada vez que ejecutas `DriverManager.getConnection()`, ocurren los siguientes pasos en la red:

- **Apertura de Socket TCP/IP:** Negociación de red (SYN, SYN-ACK, ACK).
- **Autenticación y Criptografía:** Handshake SSL/TLS y validación de contraseñas.
- **Proceso Dedicado en Servidor:** En Oracle, se crea un proceso servidor en Linux/Unix que consume memoria RAM fija (PGA).
- **Destrucción de Recursos:** El cierre (close) exige liberar búferes y cerrar sockets.

## ★ EL ESCENARIO DE COLAPSO EN PRODUCCIÓN

Si tu aplicación web recibe 200 peticiones concurrentes por segundo e intentas abrir 200 conexiones con `DriverManager`:

- El tiempo de respuesta se degrada de 10ms a varios segundos.
- La CPU del servidor de base de datos se satura gestionando handshakes en vez de procesar consultas SQL.
- Oracle alcanza el límite `MAX_PROCESSES` y lanza el temido error `ORA-00020: maximum number of processes exceeded`.

## 💡 ANALOGÍA COTIDIANA: LA FLOTA DE TAXIS VS COMPRAR UN COCHE

Si necesitas hacer un recado por la ciudad, no te compras un coche en un concesionario, lo matriculas, haces el seguro y luego lo achataras al llegar a tu destino. En su lugar, tomas un taxi de la parada que ya está arrancado y disponible, lo usas durante el trayecto y lo dejas para que otro cliente lo use inmediatamente.

## 12. HikariCP: El Estándar de la Industria

Arquitectura de piscina de conexiones pre-calentadas de cero latencia

### Dependencia Maven

COM.ZAXXER.HIKARICP

```
<dependency>
  <groupId>com.zaxxer</groupId>
  <artifactId>HikariCP</artifactId>
  <version>5.1.0</version>
</dependency>
```

HikariCP es el connection pool por defecto en Spring Boot y el más rápido del ecosistema Java gracias a optimizaciones a nivel de bytecode y cerrojos sin contención.

### Configuración Profesional de la Fuente

HIKARICONFIG &  
HIKARIDATASOURCE

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:oracle:thin:@localhost:1521:xe");
config.setUsername("c##dam2_user");
config.setPassword("dam2_pass");

config.setMaximumPoolSize(10);
config.setMinimumIdle(2);
config.setConnectionTimeout(30000); // 30s
config.setIdleTimeout(600000);       // 10min

HikariDataSource ds = new HikariDataSource(config);
```

#### 🔗 ¿QUÉ HACE `CONN.CLOSE()` AL USAR HIKARICP?

Cuando obtienes una conexión con `ds.getConnection()` y luego invocas `conn.close()` en el bloque *try-with-resources*, **la conexión física NO se destruye**. El proxy de HikariCP intercepta la llamada y devuelve la conexión a la piscina en estado *IDLE* para ser reutilizada inmediatamente en microsegundos.

## 13. Separación de Responsabilidades: DAO y DTO

Desacoplar la interfaz gráfica y la lógica de negocio de las sentencias SQL

### 1. Modelo / DTO

DATA TRANSFER OBJECT

Contenedor inmutable de datos o Java Record. No tiene lógica de negocio ni sentencias SQL:

```
public record EmpleadoDTO(
    Long id,
    String nombre,
    String email,
    BigDecimal salario
) {}
```

### 2. Interfaz DAO

CONTRATO DE PERSISTENCIA

Define qué operaciones de datos se pueden realizar sin comprometer el motor subyacente:

```
public interface EmpleadoDAO {
    void create(EmpleadoDTO emp);
    Optional<EmpleadoDTO> findById(Long id);
    List<EmpleadoDTO> findAll();
    void update(EmpleadoDTO emp);
    void delete(Long id);
}
```

### 3. DAO Impl

IMPLEMENTACIÓN JDBC

Contiene el SQL real y las llamadas JDBC. Inyecta el DataSource y maneja excepciones:

```
public class EmpleadoDAOJdbc
    implements EmpleadoDAO {
    private final DataSource ds;
    public EmpleadoDAOJdbc(DataSource ds) {
        this.ds = ds;
    }
    // Implementación métodos...
}
```

#### ✦ ¿POR QUÉ PROGRAMAR CONTRA INTERFACES DAO?

Si el día de mañana la empresa migra de Oracle a MongoDB o a un servicio REST API, la lógica de negocio y las pantallas no sufren ningún cambio; únicamente se escribe una nueva implementación `EmpleadoDAOMongo` que satisfaga el mismo contrato `EmpleadoDAO`.

## 14. Implementación Canónica de un DAO

Tratamiento de excepciones personalizadas y mapeo limpio de ResultSet a Entidad

### Método findById con Optional

JAVA 21

```
@Override
public Optional<EmpleadoDTO> findById(Long id) {
    String sql = "SELECT id, nombre, email, salario FROM emp WHERE id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return Optional.of(mapResultSet(rs));
            }
            return Optional.empty();
        }
    } catch (SQLException e) {
        throw new PersistenceException("Error al consultar empleado " + id, e);
    }
}
```

### Método Helper de Mapeo DRY (DON'T REPEAT YOURSELF)

```
private EmpleadoDTO mapResultSet(ResultSet rs) throws SQLException {
    return new EmpleadoDTO(
        rs.getLong("id"),
        rs.getString("nombre"),
        rs.getString("email"),
        rs.getBigDecimal("salario")
    );
}
```

Centralizar el mapeo de columnas en un método privado garantiza que un cambio en la tabla solo requiera actualizar una única línea de código en todo el DAO.

### ENMASCARAMIENTO DE EXCEPCIONES: PERSISTENCEEXCEPTION

Nunca propagues `SQLException` hacia las capas superiores (servicios o controladores web). Es una excepción chequeada muy acoplada a JDBC. El DAO debe capturarla y relanzar una excepción unchecked de dominio como `PersistenceException`, protegiendo la arquitectura.

## 15. Anatomía de una SQLException

Extrayendo el código de error nativo del motor y el estándar SQLState

### Métodos de Diagnóstico de SQLException

JAVA.SQL.SQLException

```
try {
    // Operación JDBC...
} catch (SQLException e) {
    System.err.println("Mensaje: " + e.getMessage());
    System.err.println("SQLState: " + e.getSQLState());
    System.err.println("Código Error Oracle: " + e.getErrorCode());

    if (e.getErrorCode() == 1) { // ORA-00001
        System.err.println("¡Violación de clave primaria o única!");
    } else if (e.getErrorCode() == 2292) { // ORA-02292
        System.err.println("¡No se puede borrar: clave foránea referenciada!");
    }
}
```

### Códigos de Error Habituales en Oracle

TOP ERRORES  
ORA

| CÓDIGO    | IDENTIFICADOR              | CAUSA HABITUAL                                        |
|-----------|----------------------------|-------------------------------------------------------|
| ORA-00001 | unique constraint violated | Inserción de clave primaria o valor unique duplicado. |
| ORA-02291 | integrity constraint (FK)  | Clave foránea apunta a un registro padre inexistente. |
| ORA-01017 | invalid username/password  | Credenciales erróneas o usuario bloqueado.            |
| ORA-12541 | TNS:no listener            | El servicio de base de datos Oracle no está iniciado. |



## 16. Errores Típicos en Exámenes y Prácticas

Los 4 descuidos más comunes al programar con JDBC y cómo evitarlos

### 1. Olvido del `rs.next()`

EXHAUSTED RESULTSET

El cursor de un `ResultSet` apunta inicialmente **antes de la primera fila**. Si intentas llamar directamente a `rs.getString("col")` sin hacer previamente un `if (rs.next())` o `while (rs.next())`, la aplicación lanzará un error de cursor no posicionado.

### 2. Fuga de Conexiones (Connection Leak)

POOL AGOTADO

No usar *try-with-resources* para cerrar conexiones o sentencias. Al cabo de 10 consultas, la piscina de conexiones se queda sin recursos disponibles (*pool empty*) y la aplicación se congela indefinidamente esperando que se libere una conexión.

### 3. Olvidar `conn.setAutoCommit(true)`

CONEXIÓN SUCIA

Si devuelves al pool una conexión con `autoCommit = false` sin haber hecho `rollback` o sin restaurar el flag a `true` en el bloque `finally`, la siguiente consulta de otro hilo heredará ese estado transaccional sucio.

### 4. Fechas obsoletas de `java.util.Date`

API ANTICUADA

Usar `java.util.Date` en lugar de la moderna API `java.time` (`LocalDate`, `LocalDateTime`). Utiliza siempre `ps.setObject(i, localDate)` y `rs.getObject(col, LocalDate.class)`.

## 17. Resumen de Clases e Interfaces Clave

Matriz de referencia rápida para el desarrollo con JDBC en Java 21

| COMPONENTE               | TIPO     | PROPÓSITO PRINCIPAL                                           | MÉTODOS CRÍTICOS                                                                                         |
|--------------------------|----------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Connection</b>        | Interfaz | Sesión activa con el motor RDBMS y delimitador transaccional. | <code>prepareStatement()</code> , <code>commit()</code> , <code>rollback()</code> , <code>close()</code> |
| <b>PreparedStatement</b> | Interfaz | Sentencia SQL precompilada con parámetros tipados ('?').      | <code>setXxx()</code> , <code>executeQuery()</code> , <code>executeUpdate()</code>                       |
| <b>CallableStatement</b> | Interfaz | Ejecución de procedimientos y funciones almacenadas (PL/SQL). | <code>registerOutParameter()</code> , <code>execute()</code> , <code>getXxx()</code>                     |
| <b>ResultSet</b>         | Interfaz | Cursor sobre la tabla de resultados de una consulta SELECT.   | <code>next()</code> , <code>getXxx()</code> , <code>wasNull()</code> , <code>getMetaData()</code>        |
| <b>HikariDataSource</b>  | Clase    | Piscina de conexiones precalentadas de alta velocidad.        | <code>getConnection()</code> , <code>close()</code>                                                      |
| <b>DatabaseMetaData</b>  | Interfaz | Catálogo global del motor de base de datos y soporte técnico. | <code>getTables()</code> , <code>getColumns()</code> , <code>getPrimaryKeys()</code>                     |

### 💡 TODAS IMPLEMENTAN AUTOCLOSEABLE

Todas las interfaces de JDBC heredan de `java.lang.AutoCloseable`. La regla de oro es declarar **siempre** las conexiones, statements y resultsets dentro de los paréntesis del bloque try (...).

## 18. Vinculación Síncrona: Cuaderno de Aula

Correspondencia 1 a 1 entre teoría impartida y prácticas en el IDE

### Bloque 1: Conectividad y CRUD (Actividades 2.1 a 2.4)

PARTE  
1

- **Actividad 2.1:** Conexión robusta a Oracle 23c con Driver OJDBC11.
- **Actividad 2.2:** Operaciones CRUD con PreparedStatement y claves autogenerated.
- **Actividad 2.3:** Navegación de ResultSet y gestión limpia de valores nulos.
- **Actividad 2.4:** Invocación de Procedimientos PL/SQL con parámetros IN y OUT.

### Bloque 2: Transacciones, Pools y DAO (Actividades 2.5 a 2.8)

PARTE  
2

- **Actividad 2.5:** Transferencia bancaria transaccional con Commit, Rollback y Savepoints.
- **Actividad 2.6:** Extracción dinámica de metadatos (DatabaseMetaData e inspector de tablas).
- **Actividad 2.7:** Configuración de Pool HikariCP y prueba de carga concurrente.
- **Actividad 2.8:** Arquitectura completa de persistencia con patrón DAO y Java Records.



#### DINÁMICA DE TRABAJO EN LAS SESIONES

Cada diapositiva teórica finaliza con una llamada a la acción hacia tu **Cuaderno de Actividades de Aula**. Al concluir la explicación, abrimos Eclipse IDE / Maven y resolvemos juntos la actividad guiada para fijar el concepto en el ordenador.

# ¡Manos a la Obra en el Aula!

Abre tu entorno de desarrollo Eclipse IDE, verifica tu contenedor o servicio Oracle Database y comencemos con la primera actividad práctica guiada.

 **03-Ejercicios/UT2/01-Cuaderno-Actividades/AD-UT2-Cuaderno-Actividades-Aula.pdf**

PROFESOR TITULAR

**Sergio Capdevila Díez**

CONTACTO Y CONSULTAS

**sergiocdi@cifpaviles.es**